Primer acercamiento al Stolen Code parte 2- [por SyXe'05]

Programa:	Useful File Utilities v2.3.5
Protección:	ASProtect v1.2x - 1.3x [Registered] + Opciones varias
Objetivos:	Desempaketar el archivo ejecutable.
URL descarga:	http://DondeSea.com
Dificultad:	Fácil/Media
Herramientas:	OllyDbg v1.10 + PEiD v0.93 + ImpREC v1.6 + ConTEXT v0.98 + Hex WorkShop 4.23 + PEditor v1.7 + LordPE Deluxe + ToPo v1.2
Cracker:	SyXe'05 .::[CracksLatinoS]::. Fecha: 20-08-2006

Un saludo a todo CracksLatinoS!

Aquí estamos con la segunda parte del tute, en la primera vimos cómo desviar dos secciones del SCode al ejecutable, además de recuperar la iat y la tabla de saltos, las direcciones donde salvamos la tabla y las secciones citadas se obtuvieron con el método de ensayo y error, cambiando las salidas de VAlloc cuando EAX==0x02F20000 y EAX==0x02F30000 en el ejecutable protegido y funcionó, lo que nos quedaría ahora será injertar parte de la pila pues los 4000h bytes no son necesarios además de injertar otras dos secciones del SCode que nos harán falta, así como solventar posibles errores que se pudieran presentar a lo largo del desempakado y que por su puesto aún no conocemos.

Recordemos que tenemos el programa parado en el OEP, si no hemos cerrado el ***ImpREC*** perfecto y sino lo abrimos, pulsamos en ATTACH TO AN ACTIVE PROCESS y elegimos "useful file utilities.exe (0xHandle)" donde ***Handle*** es variable, después picamos en LOAD TREE y abrimos tute.txt o iat.txt que es la iat que habíamos salvado, después pulsamos FIX DUMP y nos dejará un archivo llamado tute_.exe con la iat totalmente reparada pero que si lo ejecutamos no funciona claro ;(

El motivo es evidente, nos quedan por injertar dos secciones más el stack:

```
 1.- Sección [01290000] - Tamaño: 2C000h bytes
 2.- Sección [014C0000] - Tamaño: 16000h bytes
 3.-   Stack [0012C000] - Tamaño:  4000h bytes    /* injertaremos 0x2B000h */
```

Si echamos un vistazo a las secciones del EXE empakado vemos:

Address	Size	Owner	Section	Contains	Type	Access	Initial
00400000	00001000	Useful_F		PE header	Imag	R	RWE
00401000	000D0000	Useful_F		code	Imag	R	RWE
004D1000	00004000	Useful_F		data	Imag	R	RWE
004D5000	00002000	Useful_F			Imag	R	RWE
004D7000	00003000	Useful_F			Imag	R	RWE
004DA000	00001000	Useful_F			Imag	R	RWE
004DB000	00001000	Useful_F			Imag	R	RWE
004DC000	0000E000	Useful_F			Imag	R	RWE
004EA000	00026000	Useful_F	.rsrc	resources	Imag	R	RWE
00510000	00031000	Useful_F	.ufu	imports,reloc	Imag	R	RWE
00541000	00001000	Useful_F	.adata		Imag	R	RWE

En total tiene 11 secciones, pero al margen de la de recursos e importaciones son todas bastante pequeñas, grrr..

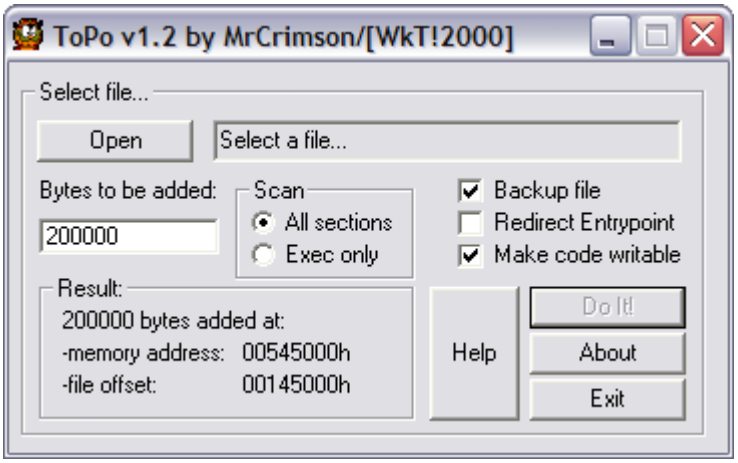
Aún así podríamos tener espacio suficiente para al menos guardar una de las dos secciones que nos quedan por injertar en la sección "imports, relocations" que tiene un tamaño mayor a las demás, y aunque la de recursos es la siguiente mayor no podemos poner nada allí porque utiliza prácticamente el 100% de su tamaño. Después de algunas pruebas vemos que podemos salvar la de 16000h bytes a partir de :0052AE00, abrimos tute_.exe en Ollydbg sin cerrar el otro Olly donde tenemos el empakado en el OEP, en este último vamos a la ventana de memoria con ALT+M y nos ponemos sobre la sección [014C0000] - 16000h bytes y hacemos DUMP IN CPU para volcar dicha sección en la ventanita Dump, seleccionamos el primer byte, picamos con el botón derecho y en el menú que aparece elegimos COPY - SELECT ALL + BINARY COPY, vamos a :0052AE00 y pegamos los bytes copiados en nuestro dumpeado:

Address	Hex dump	ASCII
0052ADE0	9A 4B 3D 16 A7 73 7D B4 98 9F 0C 5E 41 6A 6A FA	šK=▬§s}´˜Y.^Ajjú
0052ADF0	76 13 E2 EC EB DB 72 18 92 7C AE 6B CE 3E EB A7	v¶âìëÛr↑'|®kÎ>ë§
0052AE00	66 8B F2 E8 10 00 00 00 E8 01 A6 E7 94 3D 32 83	f‹òè►...è►ªç"=2f
0052AE10	00 39 7E DF 2C F5 8A FB 51 52 8B D6 5A 5E 5B 66	.9~ß,õŠûQR‹ÖZ^[f
0052AE20	8B F2 81 C3 18 57 01 00 66 BE 19 19 BF 00 00 00	‹ò□Ã↑W►.f¾↓↓¿...
0052AE30	00 8B C2 FF 34 1F B8 B7 13 74 55 59 8B D0 81 E9	.‹Âÿ4▼.¸·¶tUY‹Ð□é
0052AE40	64 A6 83 46 0F B7 C7 81 F1 CD 80 18 38 66 81 C6	d¦ƒF☼·Ç□ñÍ€↑8f□Æ
0052AE50	F2 79 81 F1 82 7D 3B 05 B6 43 89 0C 3B 81 C0 16	òy□ñ‚};♣¶C‰.;□À▬
0052AE60	7C 86 77 66 BE 33 58 81 EF 69 F1 16 31 0F B7 D7	|†wf¾3X□ïiñ▬1☼·×
0052AE70	81 C7 65 F1 16 31 0F B7 F7 81 FF A4 A9 FE FF 0F	□Çeñ▬1☼·÷□ÿ¤©þÿ☼

Debajo habían datos pero los chafamos sin más, seleccionamos todos y salvamos los cambios con COPY TO EXECUTABLE FILE - SAVE FILE y ya nos queda una sección menos, jeje.. pulsamos CTRL+F2 para recargar el EXE modificado en Olly por si tenemos que volver a escribir que lo podamos salvar con el mismo nombre, de no hacerlo no nos dejará y tendremos que ponerle otro. Vamos por faena ;)

Entrando a matar..

Lo siguiente es abrir ***ToPo v1.2*** y crear una nueva sección en el dumpeado, como hay que injertar todavía una sección de tamaños de 0x2C000h bytes, además del propio injerto, y posibles imprevistos la creamos de 200.000d bytes [decimales] con la opción CREATE A NEW SECTION (FILE SIZE INCREASED), no olvidar cerrar el Ollydbg donde lo tenemos cargado, y lo configuramos así:



Al pulsar DI IT! se escriben los valores de la parte inferior donde indica su dirección virtual y el offset donde inicia dicha sección, importante no redirigir allí el OEP aunque podríamos hacerlo lo prefiero así. Ahora volvemos a abrir el dumpeado con la nueva sección en Olly, es momento también de salvar la copia de seguridad tute_.bak que nos dejó el ToPo y renombrarla a CopiaSeg.exe por ejemplo por si algo sale mal no perder el trabajo.

Como todavía nos quedan estas dos secciones por injertar:

1.- Sección [01290000] - Tamaño: 2C000h bytes
2.- Sección [012D8000] - Tamaño: 4000h bytes

Las copiamos del original empakado que tenemos en el OEP y las pegamos en la nueva sección del dumpeado de la misma forma que hicimos antes con la otra sección, la primera [01290000] la pegamos a partir de :00545100 y la segunda [012D8000] a partir de :00571100 y salvamos nuevamente los cambios, y pulsamos CTRL+F2 para recargarlo en el Olly, quedando así:

1.- Sección [01290000] - Tamaño: 2C000h bytes - Destino: [00545100]
2.- Sección [012D8000] - Tamaño: 4000h bytes - Destino: [00571100]

Y ya nos queda menos, vamos a por la pila.. comienza a partir de :0012C000 y tiene un tamaño de 4000h bytes, pero como parte de la sección coincide con la del dumpeado la injertaremos a partir de :0012D500 que es donde empieza a cambiar, como son solo 0x2B00h bytes los podemos colocar a partir de :004E1000 perfectamente:

Address	Hex dump	ASCII
004E0FE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E0FF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1000	00 00 00 00 00 00 00 00 30 3B 14 00 99 40 F7 77	.....0;¶.™@÷w
004E1010	00 39 14 00 38 27 F5 77 C7 27 F5 77 1A 02 00 00	.9¶.8'õwÇ'õw→..
004E1020	00 00 14 00 01 00 00 00 00 00 00 00 00 00 00 00	..¶.☺.....
004E1030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E1090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E10A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
004E10B0	00 00 00 00 28 3B 14 00 40 06 14 00 00 00 00 00	....(;¶.@►¶.....
004E10C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Seleccionamos todos los bytes y salvamos los cambios, recargamos el EXE en Olly y una cosita menos ;) Y ahora el injerto:

Address	Hex dump	Disassembly		Comment
00545000	6A 04	push	4	; parametros
00545002	68 00300000	push	3000	; de VAlloc
00545007	68 00600100	push	16000	; tamanyo seccion
0054500C	68 00004C01	push	14C0000	; y su direccion
00545011	FF15 48A84D00	call	[<&kernel32.VirtualAlloc>]	; pedimos espacio
00545017	B9 00600100	mov	ecx,16000	; bytes a copiar
0054501C	BE 00AE5200	mov	esi,0052AE00	; direccion origen
00545021	BF 00004C01	mov	edi,14C0000	; direccion destino
00545026	F3:A4	rep	movsb	; copiamos bytes
00545028	90	nop		
00545029	90	nop		
0054502A	6A 04	push	4	; parametros
0054502C	68 00300000	push	3000	; de VAlloc
00545031	68 00C00200	push	2C000	; tamanyo seccion
00545036	68 00002901	push	1290000	; y su direccion
0054503B	FF15 48A84D00	call	[<&kernel32.VirtualAlloc>]	; pedimos espacio
00545041	B9 00C00200	mov	ecx,2C000	; bytes a copiar
00545046	BE 00515400	mov	esi,00545100	; direccion origen
0054504B	BF 00002901	mov	edi,1290000	; direccion destino
00545050	F3:A4	rep	movsb	; copiamos bytes
00545052	90	nop		
00545053	90	nop		
00545054	6A 04	push	4	; parametros
00545056	68 00300000	push	3000	; de VAlloc
0054505B	68 00400000	push	4000	; tamanyo seccion
00545060	68 00802D01	push	12D8000	; y su direccion
00545065	FF15 48A84D00	call	[<&kernel32.VirtualAlloc>]	; pedimos espacio
0054506B	B9 00400000	mov	ecx,4000	; bytes a copiar
00545070	BE 00115700	mov	esi,00571100	; direccion origen
00545075	BF 00802D01	mov	edi,12D8000	; direccion destino
0054507A	F3:A4	rep	movsb	; copiamos bytes
0054507C	90	nop		
0054507D	90	nop		
0054507E	B9 002B0000	mov	ecx,2B00	; bytes a copiar
00545083	BE 00104E00	mov	esi,004E1000	; direccion origen
00545088	BF 00D51200	mov	edi,12D500	; direccion destino
0054508D	F3:A4	rep	movsb	; copiamos bytes
0054508F	90	nop		
00545090	90	nop		
00545091	33C0	xor	eax,eax	;
00545093	B9 9E024C01	mov	ecx,14C029E	;
00545098	BA 0403FE7F	mov	edx,7FFE0304	; En esta parte
0054509D	BB BFD8833D	mov	ebx,3D83D8BF	; restauramos los
005450A2	BE D0032C01	mov	esi,12C03D0	; registros como
005450A7	BF 00002901	mov	edi,1290000	; en el original
005450AC	BD 78FF1200	mov	ebp,12FF78	;
005450B1	BC 74FF1200	mov	esp,12FF74	;
005450B6	90	nop		
005450B7	90	nop		
005450B8	C705 00104000	mov	dword ptr [<ModuleEntryPoint	; restauramos el
005450C2	66:C705 041040	mov	word ptr [401004],703	; salto al injerto
005450CB	68 9E024C01	push	14C029E	; direccion del OEP
005450D0	C3	ret		; saltamos alli
005450D1	90	nop		
005450D2	90	nop		

Muy simple, lo que hace básicamente es solicitar los tres bloques de memoria que se ejecutan fuera del EXE y que forman parte del SCode, luego los rellena con los bytes originales que están guardados dentro del propio ejecutable, injertamos el espacio de pila necesario, restauramos los registros para que queden igual que en el original, deshacemos el salto al injerto [que ahora veremos] y salta al OEP que sabemos que está en :014C029E, así de simple..

Primero salvamos los cambios, lo recargamos y insertamos un salto al injerto en :00401000, por eso quería el OEP provisional allí:

```
Address   Hex dump     Disassembly                Comment
0040103A  68 00505400  push    00545000           // salto al injerto
0040103F  C3           ret
00401040  0108         add     [eax],ecx
00401042  53           push    ebx
00401043  6D           insd                       I/O command
00401044  61           popad
```

Salvamos los cambios y recargamos [CTRL+F2], si nos fijamos en el injerto hemos restaurado estas líneas, de no hacerlo fallará más tarde lo he comprobado, como de momento no queda nada más es momento de probarlo, para asegurarnos de que el injerto funciona lo traceamos cuidadosamente para detectar posibles fallos, pero parece ir bastante bien, al llegar al RET salta a la zona [014C0000] como teníamos previsto, si traceamos un poco vemos que los "CALL 02F20000" se han transformado en "CALL 004DC000", jeje.. buena señal:

```
Address   Hex dump     Disassembly                Comment
014CA6B4  68 378C4C01  push    14C8C37
014CA6B9  E8 421901FF  call    tute_.004DC000
014CA6BE  EB 01        jmp     short 014CA6C1
014CA6C0  F0:334424 08 lock xor eax,[esp+8]       LOCK prefix is not allowed
014CA6C5  B8 BA484400  mov     eax,4448BA
014CA6CA  8B06         mov     eax,[esi]
014CA6CC  68 64C84C01  push    14CC864
014CA6D1  E8 2A1901FF  call    tute_.004DC000
014CA6D6  8D444B 29    lea     eax,[ebx+ecx*2+29]
014CA6DA  83E8 29      sub     eax,29
014CA6DD  64:EB 02     jmp     short 014CA6E2     Superfluous prefix
```

Pero un poco después falla, grrr.. comienzan los problemas, pasa algo rarísimo, resulta que las dos secciones que copiamos al principio [02F20000] y [02F30000] han sido modificadas, veamos una comparación:

Antes:

```
Address   Hex dump     Disassembly                Comment
004DC000  F2:          prefix repne:              Superfluous prefix
004DC001  EB 01        jmp     short 004DC004
004DC003  0F539C2E EB01C rcpps   xmm3,[esi+ebp+83C701EB]
004DC00B  C3           ret
004DC00C  00EB         add     bl,ch
004DC00E  02CD         add     cl,ch
004DC010  2013         and     [ebx],dl
004DC012  D983 EC20C1C3 fld     dword ptr [ebx+C3C120EC]
004DC018  79 83        jns     short 004DBF9D
004DC01A  CB           retf                       Far return
004DC01B  75 8B        jnz     short 004DBFA8
004DC01D  DC3E         fdivr   qword ptr [esi]
004DC01F  EB 02        jmp     short 004DC023
```

Después:

```
Address   Hex dump     Disassembly                Comment
004DC000  EB 02        jmp     short 004DC004
004DC002  CD 20        int     20
004DC004  51           push    ecx
004DC005  9C           pushfd
004DC006  334C24 08    xor     ecx,[esp+8]
004DC00A  83E9 7D      sub     ecx,7D
004DC00D  F3:          prefix rep:                Superfluous prefi
004DC00E  EB 02        jmp     short 004DC012
004DC010  CD 20        int     20
004DC012  83EC 20      sub     esp,20
004DC015  2E:EB 01     jmp     short 004DC019     Superfluous prefi
004DC018  9A 2B4C2408 C1 call  far C9C1:08244C2B  Far call
004DC01F  ED           in      eax,dx             I/O command
```

Es muy extraño como antes comenzaba con 0xF2EB01.. y ahora con 0xEB02CD.. pues resulta que fueron encriptadas con el PID actual, las volvemos a seleccionar del original y la pegamos encima, esto hay que hacerlo con estas dos secciones:

1.- Sección [02F20000] - Tamaño: 1000h bytes - Destino: [004DC000]
2.- Sección [02F30000] - Tamaño: 1000h bytes - Destino: [004DE000]

Como entre sección y sección hay un espacio lleno de ceros que no influyen copiamos desde :004DC000 hasta :004DEFFF y pegamos estos bytes en el dumpeado chafando los erróneos que haya que como se puede ver son muchos:

```
Address   Hex dump                                          ASCII
004DC000  F2 EB 01 0F 53 9C 2E EB 01 C7 83 C3 00 EB 02 CD  òë ►♫Sœ.ë►Çƒà.ë►Í
004DC010  20 13 D9 83 EC 20 C1 C3 79 83 CB 75 8B DC 3E EB   ‼Ùƒì ÁÃyƒËu‹Ü>ë
004DC020  02 CD 20 51 8F 43 04 81 C9 60 70 C7 1C F2 EB 01  ►Í QÇC►♦É`pÇ└òë►
004DC030  E9 89 53 08 BA 3A 65 46 00 BA B2 98 41 00 57 8F  é‰S◘º:eF.º²˜A.W
004DC040  43 1C 23 FF 89 43 00 83 D8 BF 03 44 24 18 F3 EB  C#y‰C.ƒØ¿♥D$↑óë
004DC050  02 CD 20 55 8F 44 23 14 F2 EB 01 0F BD 6A A2 44  ►Í UÇD#¶òë►♫½j¢D
004DC060  00 BD E2 FA 42 00 F3 EB 02 CD 20 89 73 18 83 DE  .½âúB.óë►Í ‰s↑ƒÞ
004DC070  A9 F2 EB 01 0F BE 76 96 42 00 33 4C 24 28 C1 D1  ©òë►♫¾v–B.3L$(ÁÑ
004DC080  19 C1 C9 51 B9 9A C1 44 00 8D 4C 24 25 83 E9 25  ↓ÁÉQ'šÁD.ÌL$%ƒé%
004DC090  83 C1 2C 51 0B CB 53 33 4C 24 28 B9 F2 11 46 00  ƒÁ,Q♂ËS3L$('ò◄F.
004DC0A0  53 81 D1 3C 5E D4 A3 59 F2 EB 01 9A 36 EB 01 9A  S◘Ñ<^ÔƒYòë►šëë►š
004DC0B0  8D 4C 21 20 8B 09 51 81 E1 EC 3A A5 70 33 4C 24  ÌL! ‹.Q◘áì:¥p3L$
004DC0C0  28 33 4C 24 08 53 8D 4C 4B 37 8D 4C 21 C9 59 83  (3L$◘SÌLK7L!ÉYƒ
004DC0D0  C1 28 8B 09 8D 4C 21 FB 51 8D 49 2F 8D 8C 07 4C  Á(‹.Ì!ûQÌI/Ì♠•L
```

Volvemos a salvar los cambios, volvemos a probar el ejecutable y parece que funciona mejor, traceamos la zona del ***SCode*** y al llegar al salto indirecto que nos enviaba a :02F30000 vemos que ahora nos lleva a :004DE000, jeje..

```
Address   Hex dump      Disassembly                Comment
012ABC3F    8B45 08     mov     eax,[ebp+8]
012ABC42    E8 89F4FFFF call    012AB0D0
012ABC47    FF75 FC     push    dword ptr [ebp-4]
012ABC4A    FF75 10     push    dword ptr [ebp+10]
012ABC4D    FF75 0C     push    dword ptr [ebp+C]
012ABC50    8B45 F8     mov     eax,[ebp-8]
012ABC53  - FF60 24     jmp     [eax+24]           tute_.004DE000
012ABC56  v EB 01       jmp     short 012ABC59
012ABC58    9A 5F5E5B8B E5 call far 5DE5:8B5B5E5F  Far call
012ABC5F    C2 0C00     ret     0C
```

Una vez allí llegamos al otro salto indirecto:

```
004DE0CC    9D              popfd
004DE0CD    5C              pop     esp
004DE0CE  - FF6424 FC       jmp     [esp-4]
```

Que nos envía de nuevo a la zona del ***SCode*** cumpliéndose el ciclo lo que indica que el injerto ha funcionado bien hasta ahora, cuando parece que hemos terminado el trabajo se nos presenta otro reto porque vuelve a fallar aquí:

```
Address   Hex dump      Disassembly                Comment
004044D6  v 7E 14       jle     short 004044EC
004044D8    8B04DF      mov     eax,[edi+ebx*8]    // lee valor maldito..
004044DB    43          inc     ebx
004044DC    891D 40564D00 mov   [4D5640],ebx
004044E2    85C0        test    eax,eax
004044E4  v 74 02       je      short 004044E8
004044E6    FFD0        call    eax                // Aqui falla..
004044E8    3BF3        cmp     esi,ebx
004044EA  ^ 7F EC       jg      short 004044D8
004044EC    33C0        xor     eax,eax
```

Ese "CALL EAX" salta a una dirección que no existe en nuestro dumpeado y sí en el original, se trata de :012AC580 y estamos en la famosa ***Init Table***, un poco antes lee dicho valor del dump, nos ponemos en :004044D8 y hacemos un FOLLOW IN DUMP - MEMORY ADDRESS para ver dicho valor que nos da problemas:

```
Address   Hex dump                                          ASCII
004D0948  80 C5 2A 01 AC C5 2A 01 3C 44 CA 75 07 E3 65 E3  €Å* ¬Å* <DÊu•ãeã
004D0958  42 25 19 04 7D 09 84 B0  Más problemas..  E 55  B%↓♦}.„°œf┼ïP@.U
004D0968  1B 23 69 88 C4 68 50 17 60 F7 93 11 66 91 C6 38  ←#iˆÄhP↨`÷"◄f'Æ8
004D0978  78 63 B6 AE  Problemas :( A6 02 EF 19 B3 77 E9  xc¶®È°@]]¦►ï↓*wé
004D0988  C8 1C 43 DF 7C F9 B4 01 00 6D D8 A7 44 BA A3 47  ÈCß|ù´►.mØ§D°£G
004D0998  EF D2 F0 C1 EF D1 7D 58 E9 F9 F8 FE 00 9D 8C C5  ïÒðÁïÑ}Xéùøþ.ÌÅ
004D09A8  E7 5C 58 DE EB B2 5C A8 FB 79 97 CF A1 DE 27 85  ç\XÞë²\¨ûy—Ï¡Þ'…
```

No hay sólo uno sino que hay hasta dos, aunque creo que el segundo no lo utiliza. Aprovechando que todavía tenemos el otro Olly abierto con el original parado en el OEP ponemos un BP en el "CALL EAX" y pulsamos F9 para caer en él, entramos con F7 y echamos un vistazo a ver qué hay más allá..

No hay mucha cosa, unas pocas instrucciones y enseguida un salto indirecto:

```
012AC594    A1 445B2B01     mov     eax,[12B5B44]
012AC599    50              push    eax
012AC59A    A1 405B2B01     mov     eax,[12B5B40]
012AC59F    50              push    eax
012AC5A0    FF15 485B2B01   call    [12B5B48]           // a :012C0454
012AC5A6    8BC3            mov     eax,ebx
012AC5A8    5B              pop     ebx
012AC5A9    C3              ret
```

Que como indica el comentario nos pone en :012C0454, si traceamos la zona nos llevamos una desagradable sorpresa pues al llegar aquí:

```
Address   Hex dump      Disassembly                  Comment
012C057F    F3:         prefix rep:                  Superfluous prefix
012C0580  v EB 02       jmp     short 012C0584
012C0582    CD 20       int     20
012C0584    FFD0        call    eax                  // Aqui falla
012C0586    FF7424 20   push    dword ptr [esp+20]
012C058A    23C5        and     eax,ebp
012C058C    58          pop     eax
012C058D    81F3 B0328552 xor   ebx,528532B0
012C0593    5B          pop     ebx
012C0594    BA 62234900 mov     edx,492362
012C0599    BA 5A584400 mov     edx,44585A
012C059E    5A          pop     edx
012C059F    F3:         prefix rep:                  Superfluous prefix
012C05A0  v EB 02       jmp     short 012C05A4
```

Resulta que EAX==0xFF646CBB y al hacer "CALL EAX" genera un error ACCESS_VIOLATION, he revisado la rutina y no veo nada raro, ni llamadas a *GetTickCount* ni nada de nada, todavía no sé cómo nos detecta pero lo hace ¿?

Una solución que yo encontré fue reiniciar Olly, llegar al OEP con el script que vimos antes, poner un HBP en el "CALL EAX" e ir anotando el valor de EAX en un papel para cada una de las pasadas del loop donde está dicho CALL, son éstos:

```
00406D98    00429C88    00438AB8    0046FE6C    00490350    004BF078    004A6C74
00406E3C    0042B990    00438C30    0046B358    004913C4    004C44EC    004A96E0
00406BEC    0042A8D8    0042B9E8    0046A88C    00491944    004A0E0C    004B7030
00406E3C    0042A158    0042C67C    0046F764    00493F84    004A1BFC    004B8A44
00407C40    004205B8    00438A60    0046FD90    004946A4    004A1C60    004B5014
00407C78    004295E8    00438C78    0046E6F4    00496A90    004979C0    004A9BE4
00413B9C    00456688    0043F8A8    0046E74C    00496C64    00496FD4    004AA1B8
00407F70    0042A918    0043F854    0046FDFC    0049700C    004AA69C    004AA9EC
0040E810    0045CACC    00441650    0046FE34    0049F0BC    004BC554    004ABE58
0040E920    00453A0C    00441D14    0046B420    004CE61C    004BEC48    004ACCB0
004129A4    00454CEC    00441C8C    004826E0    004CCD44    004A1C98    004B3B60
00412D28    00442ADC    00441E7C    004830F8    0049F0F4    004A1CF0    004D0434
00413AF4    00438B60    0042C994    004851FC    00497C80    004A2BAC    0049E020
004201FC    00438B28    0046A344    0048E4BC    004C4960    004A4BE0    004C9844
00429724    00438AF0    0046A854    00490084    004A0DD4    004A6B3C    004D046C
```

Que son las direcciones de inicio de las funciones que deberán ejecutarse en la inicialización del programa, de ahí el nombre ***Init Table***. La solución es sencilla, bien estudiamos cómo se forma ésta traceando el packer o bien copiamos estos valores en una tabla para que se ejecuten en el loop que hay en el EXE sin saltar a secciones externas que es rápida y sencilla.

Antes vimos que en esta instrucción:

```
004044D8  > /8B04DF         mov     eax,[edi+ebx*8]        // lee valor maldito..
004044DB  . |43             inc     ebx
004044DC  . |891D 40564D00 mov     [4D5640],ebx
```

leía de [4D0948], pues allí es donde debemos colocar la tabla anterior:

```
Address   Hex dump                                         ASCII
004D0948  98 6D 40 00 3C 6E 40 00 EC 6B 40 00 3C 6E 40 00  ˜m@.<n@.ìk@.<n@.
004D0958  40 7C 40 00 78 7C 40 00 9C 3B 41 00 70 7F 40 00  @|@.x|@.œ;A.p⌂@.
004D0968  10 E8 40 00 20 E9 40 00 A4 29 41 00 28 2D 41 00  ►è@. é@.¤)A.(-A.
004D0978  F4 3A 41 00 FC 01 42 00 24 97 42 00 88 9C 42 00  ôºA.ü☺B.$—B.ˆœB.
004D0988  90 B9 42 00 D8 A8 42 00 58 A1 42 00 B8 05 42 00  ¹'B.Ø¨B.X¡B.¸♣B.
004D0998  E8 95 42 00 88 66 45 00 18 A9 42 00 CC CA 45 00  è•B.ˆfE.↑©B.ÌÊE.
004D09A8  0C 3A 45 00 EC 4C 45 00 DC 2A 44 00 60 8B 43 00  ♀:E.ìLE.Ü*D.`‹C.
004D09B8  28 8B 43 00 F0 8A 43 00 B8 8A 43 00 30 8C 43 00  (‹C.ðŠC.¸ŠC.0ŒC.
004D09C8  E8 B9 42 00 7C C6 42 00 60 8A 43 00 78 8C 43 00  è'B.|ÆB.`ŠC.xŒC.
004D09D8  A8 F8 43 00 54 F8 43 00 50 16 44 00 14 1D 44 00  ¨øC.TøC.P▬D.¶↔D.
```

La tabla continúa hasta cubrir todas las funciones, pero si ejecutamos el programa vemos que no las ejecuta en orden sino que va salteándolas de dos en dos, el problema está en la línea anteriormente marcada en amarillo y que modificamos así:

```
Address   Hex dump     Disassembly                    Comment
004044D8  8B049F       mov     eax,[edi+ebx*4]        // linea modificada..
004044DB  43           inc     ebx
004044DC  891D 40564D0 mov     [4D5640],ebx
004044E2  85C0         test    eax,eax
004044E4  74 02        je      short 004044E8
004044E6  FFD0         call    eax
004044E8  3BF3         cmp     esi,ebx
004044EA  7F EC        jg      short 004044D8
004044EC  33C0         xor     eax,eax
```

Cambiamos el valor de desplazamiento de 0x08h a 0x04h para que salte de una en una, recordemos que un "DWORD == 04 bytes" por esto este valor. Pero aún no hemos terminado porque ahora sólo ejecuta el bucle una vez, recordemos que el packer modificaba esta parte para desviarnos a una sección virtual desde aquí, y que se ejecutaba una sola vez, pero nosotros ahora necesitamos que se ejecute 105 veces que es el número de funciones.

La solución es también sencilla, si nos fijamos después de la instrucción que hemos modificado se incrementa EBX que es el contador del loop, este valor lo compara después del call con el de ESI que es el contador a modificar, subamos hacia arriba hasta encontrar donde se inicializa este valor:

```
Address   Hex dump     Disassembly                    Comment
004044B6  A1 3C564D00  mov     eax,[4D563C]
004044BB  85C0         test    eax,eax
004044BD  74 4B        je      short 0040450A
004044BF  8B30         mov     esi,[eax]              ESI toma su valor!
004044C1  33DB         xor     ebx,ebx
004044C3  8B78 04      mov     edi,[eax+4]
004044C6  33D2         xor     edx,edx
004044C8  55           push    ebp
004044C9  68 F6444000  push    004044F6
004044CE  64:FF32      push    dword ptr fs:[edx]
004044D1  64:8922      mov     fs:[edx],esp
004044D4  3BF3         cmp     esi,ebx
004044D6  7E 14        jle     short 004044EC
004044D8  8B049F       mov     eax,[edi+ebx*4]        // linea modificada..
004044DB  43           inc     ebx
004044DC  891D 40564D0 mov     [4D5640],ebx
004044E2  85C0         test    eax,eax
004044E4  74 02        je      short 004044E8
004044E6  FFD0         call    eax
004044E8  3BF3         cmp     esi,ebx
004044EA  7F EC        jg      short 004044D8
004044EC  33C0         xor     eax,eax
```

Ahí está en :004044BF y lo lee de [EAX] que toma su valor un poco antes de la variable [4D563C] que guarda el valor 0x4D0650 que es donde se almacena el valor CONTADOR del bucle, si vamos hasta allí con CTRL+G lo descubrimos:

```
Address   Hex dump                                        ASCII
004D0650  01 00 00 00 48 09 4D 00 19 8B 8F D2 E7 AD 11 55  ....H.M.
004D0660  32 1D B6 B6 BE D0 80 94 7D 68 07 1B 06 5F F5 A4  2¶¶
004D0670  B8 4F B7 2C Valor contador == 1  DF B1 24 66  ,O.,
004D0680  B4 1D B3 6F A9 A2 19 17 B5 AE 3C 7F 47 14 42 3D
004D0690  6D 7C 2C AD 5F D8 8E B1 26 3B 27 D1 35 A4 64 02
004D06A0  39 AB 1E 87 7A 22 1D B6 83 F8 29 84 F1 21 C2 33  9
```

Este valor lo cambiamos por 105, pero no a lo bestia, antes lo pasamos a formato hexadecimal 105d => 68h que es el valor a cambiar:

```
Address   Hex dump                                        ASCII
004D0650  68 00 00 00 48 09 4D 00 19 8B 8F D2 E7 AD 11 55  h...H.M.
004D0660  32 1D B6 B6 BE D0 80 94 7D 68 07 1B 06 5F F5 A4  2¶¶
004D0670  B8 4F B7 2C Contador modificado!  F B1 24 66  ,O.,
004D0680  B4 1D B3 6F A9 A2 19 17 B5 AE 3C 7F 47 14 42 3D
004D0690  6D 7C 2C AD 5F D8 8E B1 26 3B 27 D1 35 A4 64 02
004D06A0  39 AB 1E 87 7A 22 1D B6 83 F8 29 84 F1 21 C2 33  9
```

Salvamos ambos cambios con COPY TO EXECUTABLE - ALL MODIFICATIONS - COPY ALL - SAVE FILE y recargamos con CTRL+F2, y podemos decir que estamos un poquito más cerca, pues aún no hemos terminado y aunque llegara a funcionar cuando cerremos el programa volverá a fallar, porqué? pues porque al igual que inicializa el programa también lo desinicializa, o sea que existe una ***Deinit Table*** que debemos encontrar y reparar como hemos hecho con esta.

Buscando la Deinit Table..

A estas alturas estamos a punto de concluir nuestro trabajo, pero aún nos quedan algunos flecos, de momento nos ocuparemos de este. Una forma rápida de encontrarla es poner un BPM en la sección de código con el programa en ejecución, de esta manera descubrimos el loop donde queda corriendo en pantalla:

```
Address   Hex dump   Disassembly                      Comment
00468FA9  > 33C0     xor     eax,eax                  // inicio loop..
00468FAB  . 55       push    ebp
00468FAC  . 68 C98F4600 push  00468FC9
00468FB1  . 64:FF30  push    dword ptr fs:[eax]
00468FB4  . 64:8920  mov     fs:[eax],esp
00468FB7  . 8B45 FC  mov     eax,[ebp-4]
00468FBA  . E8 D1FDFFFF call  00468D90
00468FBF  . 33C0     xor     eax,eax
00468FC1  . 5A       pop     edx
00468FC2  . 59       pop     ecx
00468FC3  . 59       pop     ecx
00468FC4  . 64:8910  mov     fs:[eax],edx
00468FC7  .v EB 15   jmp     short 00468FDE
00468FC9  .^ E9 4AAFF9FF jmp 00403F18
00468FCE  . 8B55 FC  mov     edx,[ebp-4]
00468FD1  . 8B45 FC  mov     eax,[ebp-4]
00468FD4  . E8 4B000000 call 00469024
00468FD9  . E8 A2B2F9FF call 00404280
00468FDE  > 8B45 FC  mov     eax,[ebp-4]
00468FE1  . 80B8 9C00000 cmp byte ptr [eax+9C],0
00468FE8  .^ 74 BF   je      short 00468FA9          // al inicio del loop..
00468FEA  > 33C0     xor     eax,eax
00468FEC  . 5A       pop     edx
00468FED  . 59       pop     ecx
```

Quitamos el BPM y ponemos un BP con F2 al final del loop y cerramos el programa, Olly romperá en el BP, a partir es cuestión de tracear hasta encontrar la tabla de desinicialización, es ésta:

```
Address   Hex dump   Disassembly                      Comment
0040447A  > 4B       dec     ebx
0040447B    895F 0C  mov     [edi+C],ebx
0040447E    8B44DE 04 mov    eax,[esi+ebx*8+4]       // valor maldito..
00404482    85C0     test    eax,eax
00404484  v 74 02    je      short 00404488
00404486    FFD0     call    eax                     // Petardazo ;(
00404488    85DB     test    ebx,ebx
0040448A  ^ 7F EE    jg      short 0040447A
0040448C    33C0     xor     eax,eax
```

Haremos lo mismo de antes, llegaremos a esta zona en el EXE empakado y entraremos en el "CALL EAX" traceamos hasta llegar al siguiente "CALL EAX" que coincide con el anterior de :012C0584 y anotamos los valores otra vez:

004D0628	004A6C44	004BF048	0048E48C	0046FDAC	00438A88	00429C58
004D043C	004A6B0C	004A0DA4	004851CC	0046A824	00438AC0	004296F4
004C97DC	004A4BB0	004C4930	004830C8	0046A2E8	00438AF8	00420154
0049DF70	004A2B2C	00497C50	00482668	0042C964	00438B30	00413AA4
004D0404	004A1CC0	0049F0C4	0046B3F0	00441E4C	00442A70	00412CF8
004B3B30	004A1C68	004CCD04	00491908	00441C50	00454CBC	00412940
004ACC80	004BEC18	004CE5EC	00491394	00441CE4	004539A4	0040E8F0
004ABE00	004BC514	0049F08C	0046E71C	004415F4	0045CA5C	0040E690
004AA9BC	004AA66C	00496FDC	0046E6BC	0043F824	0042A8E8	00407F40
004AA17C	00496FA4	00496C34	0046FD48	0043F878	00456658	00413B6C
004A9BA8	00497980	00496A60	0046F71C	00438C48	00429514	00407C48
004B4FE4	004A1C30	00494668	0046A85C	00438A30	00420588	00407C10
004B8A14	004A1BCC	00493F48	0046B328	0042C640	0042A128	00406E0C
004B6FF0	004A0DDC	00490320	0046FE3C	0042B9B8	0042A8A8	00406B98
004A96B0	004C44BC	00490054	0046FE04	00438C00	0042B94C	00406D68

La cuestión ahora es lograr que se ejecuten una a una estas funciones dentro del loop [40447A..40448A] y en este mismo orden, en la dirección :004044E obtenemos la dirección de donde los busca con un FOLLOW IN DUMP - MEMORY ADDRESS y caemos aquí:

```
Address   Hex dump                                         ASCII
004D0C84  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004D0C94  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004D0CA4  00 00 00 00 lee de aqui! )0 00 00 00 00 00 00 00  ................
004D0CB4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004D0CC4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
004D0CD4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Ya sabemos donde irá dicha tabla, además está muy cerca de la otra, pero en este caso se ejecuta en orden inverso pues como se ve el contador [EBX] se decrementa en cada ciclo del loop, lo que indica que la dirección [004D0C84] es la última de la tabla, la primera la encontramos con una simple operación:

4d0c84 - 68*4 == 4d0ae4

Pero tenemos un gran problema porque si ubicamos la tabla con este inicio resulta que invadimos otras zonas que no están reservadas para ella:

```
Address   Hex dump                                          ASCII
004D0AB4  E0 96 4A 00 30 70 4B 00 44 8A 4B 00 14 50 4B 00  à-J.0pK.DŠK.¶PK.
004D0AC4  E4 9B 4A 00 B8 A1 4A 00 EC A9 4A 00 58 BE 4A 00  ä›J.¸¡J.ì©J.X¾J.
004D0AD4  B0 CC 4A 00 60 3B 4B 00 34 04 4D 00 20 E0 49 00  °ÌJ.`;K.4♦M. àI.
004D0AE4  44 98 4C 00 6C 04 4D 00 7D 2B 38 69 0F 7A B9 F6  D˜L.l♦M.}+8i☼z'ö
004D0AF4  02 A5 E3 7D B7 42 D0 3B A4 BE 23 71 7C 62 F1 52  ☻¥ã}·BÐ;¤¾#q|bñR
004D0B04  51 53 50 50 54 9A E9 AA DE FF 00 58 CB F6 B6 C9  QSPPTšéªÞÿ.XËö¶É
004D0B14  2B 1C 08 64 54 9A E9 CA 8D FF 00 89 63 1A E8 FB  +∟♦dTšéÊ¤ÿ.‰c→èû
004D0B24  B7 9A 8C 06 69 E6 C1 BF D8 1C 25 16 D6 18 98 F2  ·šŒ♠iæÁ¿Ø∟%▬Ö↑˜ò
004D0B34  28 65 0F 1F 97 0B D2 86 F7 0E 3C C9 B4 B2 1E 4D  (e☼▼—♂Ò†÷♫<É´²▲M
004D0B44  CD C3 63 31 4B 71 92 D4 A4 8C 37 E5 90 39 74 0E  ÍÃc1Kq'Ô¤Œ7å♀9t♫
004D0B54  3B 4E 2A BF E9 05 7F C5 72 B0 BC 7E 19 9C BD F2  ;N*¿é♣⌂Ár°¼~↓œ½ò
004D0B64  BE 49 67 3B 6D A9 F7 CB A6 18 0B AA 10 1E AF AE  ¾Ig;m©÷Ë¦↑♂ª►▲¯®
004D0B74  53 56 F2 2E 59 EA A8 DE 52 61 53 60 60 55 9A E9  SVò.Yê¨ÞRaS``Ušé
```

La línea ROJA delimita la ***Init Table*** y lo remarcado AZUL es la zona que invade la tabla en el supuesto de ubicarla a partir de :004D0AE4 por lo que la solución no pasa por aquí. Después de observar y tracear el empakado no le veo solución sin injertar un poco más, jeje..

La solución es fácil también, vamos a modificar la zona para que ejecute las llamadas una por una en el orden correcto, primero busco una zona amplia de ceros que no sea utilizada como buffer por el programa para guardar la tabla:

```
Address   Hex dump                                          ASCII
004D0C8C  68 6D 40 00 98 6B 40 00 0C 6E 40 00 10 7C 40 00  hm@.˜k@..n@.►|@.
004D0C9C  48 7C 40 00 6C 3B 41 00 40 7F 40 00 90 E6 40 00  H|@.l;A.@⌂@.æ@.
004D0CAC  F0 E8 40 00 40 29 41 00 F8 2C 41 00 A4 3A 41 00  ðè@.@)A.ø,A.¤:A.
004D0CBC  54 01 42 00 F4 96 42 00 58 9C 42 00 4C B9 42 00  T☺B.ô—B.XœB.L'B.
004D0CCC  A8 A8 42 00 28 A1 42 00 88 05 42 00 14 95 42 00  ¨¨B.(¡B.ˆ♣B.¶•B.
004D0CDC  58 66 45 00 E8 A8 42 00 5C CA 45 00 A4 39 45 00  XfE.è¨B.\ÊE.¤9E.
004D0CEC  BC 4C 45 00 70 2A 44 00 30 8B 43 00 F8 8A 43 00  ¼LE.p*D.0‹C.øŠC.
004D0CFC  C0 8A 43 00 88 8A 43 00 00 8C 43 00 B8 B9 42 00  ÀŠC.ˆŠC..ŒC.¸'B.
004D0D0C  40 C6 42 00 30 8A 43 00 48 8C 43 00 78 F8 43 00  @ÆB.0ŠC.HŒC.xøC.
004D0D1C  24 F8 43 00 F4 15 44 00 E4 1C 44 00 50 1C 44 00  $øC.ô§D.äD.PD.
004D0D2C  4C 1E 44 00 64 C9 42 00 E8 A2 46 00 24 A8 46 00  LD.dÉB.è¢F.$¨F.
004D0D3C  AC FD 46 00 04 FE 46 00 3C FE 46 00 28 B3 46 00  ¬ýF.♦þF.<þF.(³F.
004D0D4C  5C A8 46 00 1C F7 46 00 48 FD 46 00 BC E6 46 00  \¨F.∟÷F.HýF.¼æF.
004D0D5C  1C E7 46 00 94 13 49 00 08 19 49 00 F0 B3 46 00  ∟çF."‼I.◘↓I.ð³F.
004D0D6C  68 26 48 00 C8 30 48 00 CC 51 48 00 8C E4 48 00  h&H.È0H.ÌQH.ŒäH.
004D0D7C  54 00 49 00 20 03 49 00 48 3F 49 00 68 46 49 00  T.I. ♥I.H?I.hFI.
004D0D8C  60 6A 49 00 34 6C 49 00 DC 6F 49 00 8C F0 49 00  `jI.4lI.ÜoI.Œðl.
004D0D9C  EC E5 4C 00 04 CD 4C 00 C4 F0 49 00 50 7C 49 00  ìåL.♦ÍL.Ðð I.P|I.
004D0DAC  30 49 4C 00 A4 0D 4A 00 48 F0 4B 00 BC 44 4C 00  0IL.¤♪J.HðK.¼DL.
004D0DBC  DC 0D 4A 00 CC 1B 4A 00 30 1C 4A 00 80 79 49 00  Ü♪J.Ì←J.0∟J.€yI.
```

Seleccionamos toda la tabla y salvamos los cambios y recargamos con CTRL+F2 para no perderlos, y ahora buscamos el modo de que se ejecuten, hay que hacer notar que las direcciones han sido colocadas en orden inverso a como las habíamos anotado puesto que se ejecutarán de abajo hacia arriba y no de arriba hacia abajo como hacía el packer, de esta manera no perderemos su orden de ejecución, esta es la zona que deberemos modificar:

```
Address   Hex dump     Disassembly                    Comment
0040445B    8B47 08    mov     eax,[edi+8]
0040445E    85C0       test    eax,eax
00404460  v 74 48      je      short 004044AA
00404462    8B5F 0C    mov     ebx,[edi+C]
00404465    8B70 04    mov     esi,[eax+4]            ; Aqui ira un salto
00404468    33D2       xor     edx,edx                ; a pequenyo injerto
0040446A    55         push    ebp
0040446B    68 96444000 push   00404496
00404470    64:FF32    push    dword ptr fs:[edx]
00404473    64:8922    mov     fs:[edx],esp
00404476    85DB       test    ebx,ebx
00404478  v 7E 12      jle     short 0040448C
0040447A  > 4B         dec     ebx
0040447B    895F 0C    mov     [edi+C],ebx            ; Esto tambien
0040447E    8B44DE 04  mov     eax,[esi+ebx*8+4]      ; lo cambiaremos
00404482    85C0       test    eax,eax
00404484  v 74 02      je      short 00404488
00404486    FFD0       call    eax                    // ejecuta funcion..
00404488    85DB       test    ebx,ebx
0040448A  ^ 7F EE      jg      short 0040447A
0040448C    33C0       xor     eax,eax
```

La solución está en desviar mediante un salto la ejecución hacia una zona libre donde colocar en ESI un valor "CLAVE" que ahora veremos, en dicho injerto además se ejecutarán las sentencias sobrescritas, no obstante habrá que modificar un poco el código para que el puntero a la tabla esté en *ESI, veamos los cambios:

```
Address   Hex dump     Disassembly                    Comment
0040445B  8B47 08      mov     eax,[edi+8]
0040445E  85C0         test    eax,eax
00404460  74 48        je      short 004044AA
00404462  8B5F 0C      mov     ebx,[edi+C]
00404465  E9 72C70C00  jmp     004D0BDC               // Salto al injerto
0040446A  55           push    ebp
0040446B  68 96444000  push    00404496
00404470  64:FF32      push    dword ptr fs:[edx]
00404473  64:8922      mov     fs:[edx],esp
00404476  85DB         test    ebx,ebx
00404478  7E 12        jle     short 0040448C
0040447A  4B           dec     ebx
0040447B  83EE 04      sub     esi,4                  ; Contador-4
0040447E  90           nop                             ; no necesarias
0040447F  90           nop                             ; estas dos lineas
00404480  8B06         mov     eax,[esi]              ; leemos direccion
00404482  85C0         test    eax,eax
00404484  74 02        je      short 00404488
00404486  FFD0         call    eax                    // ejecuta funcion..
00404488  85DB         test    ebx,ebx
0040448A  7F EE        jg      short 0040447A
0040448C  33C0         xor     eax,eax
```

Lo que se ha hecho es desviar la ejecución hacia :004D0BDC antes de llegar al loop donde ejecutará las funciones, el objetivo está en poner en ESI la dirección del final de la Tabla+4 puesto que se le restarán 4 unidades [Un DWORD] antes de llegar al "CALL EAX" como la tabla termina en :004D0E2C el valor de ESI será :004DE30, además se ejecutarán las instrucciones sobrescritas por el salto:

```
Address   Hex dump     Disassembly                    Comment
004D0BDC  33D2         xor     edx,edx                ; instruccion sobrescrita
004D0BDE  BE 300E4D00  mov     esi,004D0E30           ; ESI toma su valor
004D0BE3  68 6A444000  push    0040446A               ; empujamos direccion
004D0BE8  C3           ret                            ; y retornamos..
004D0BE9  90           nop
004D0BEA  90           nop
004D0BEB  0000         add     [eax],al
004D0BED  0000         add     [eax],al
004D0BEF  0000         add     [eax],al
004D0BF1  0000         add     [eax],al
```

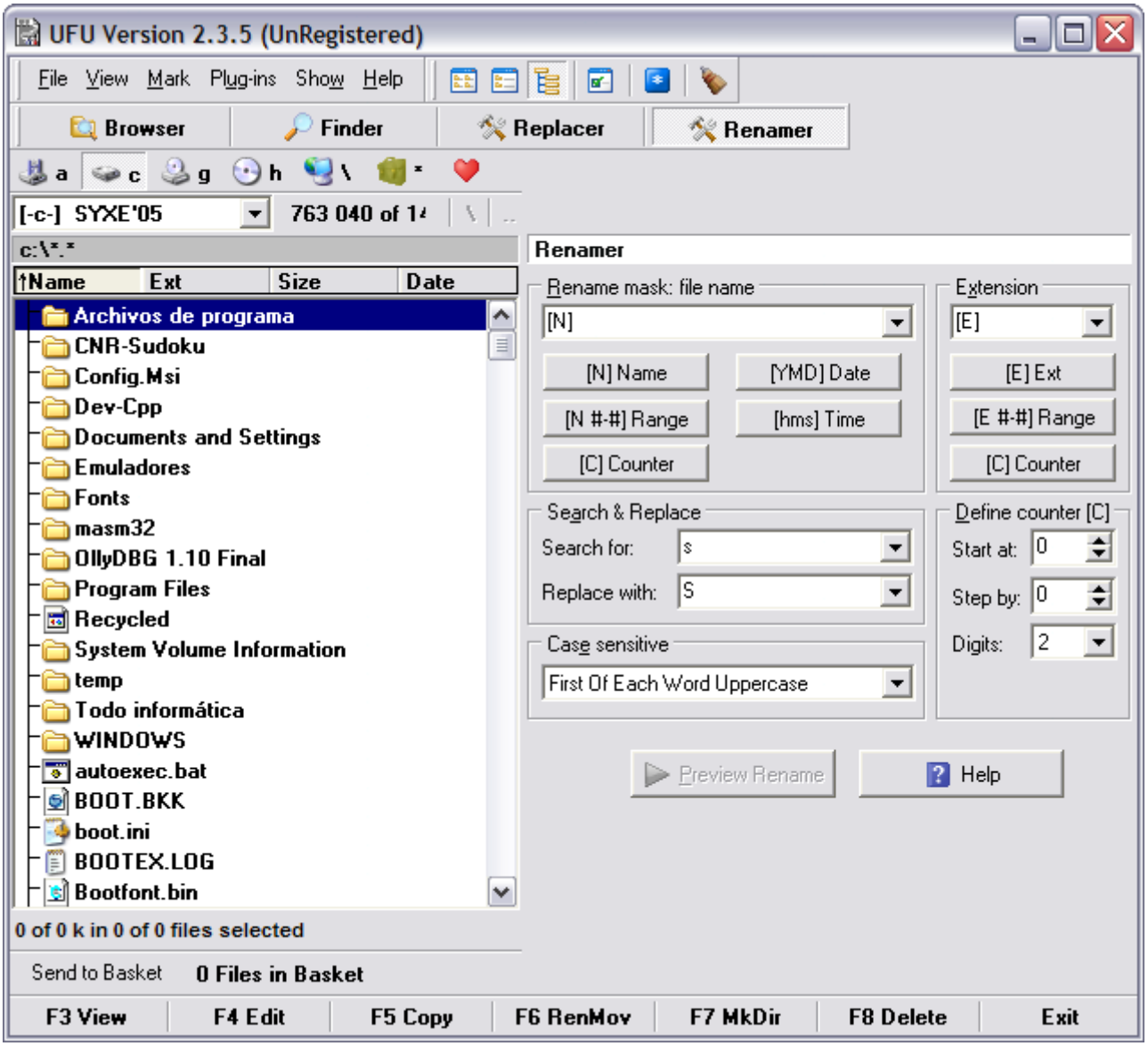
No necesita explicación creo yo, recordemos por si acaso que las líneas que reemplazó el salto a este injerto fueron estas dos:

```
00404465  8B70 04      mov esi,[eax+4]          ; No cal ejecutarla
00404468  33D2         xor edx,edx              ; Esta si
```

Lógicamente como modificamos ESI en dicho injerto no necesitaremos restaurar la primera, sí en cambio la segunda y por eso lo hacemos, luego ponemos en ESI el valor CLAVE y retorna con un PUSH - RET a la zona anterior, salvamos los cambio y recargamos Olly con CTRL+F2, ejecutamos tute_.exe y...

Aparece la nagscreen inicial que nos invita a registrarnos, esperamos tres o cuatro segundos hasta que se active el tercer botón "OK" y lo pulsamos, entre tanto saltan algunas excepciones en Olly al cargar el archivo "BatchReplacer.plgn" que según vimos PEiD nos daba ASPack v2.12 como compresor, pero que por las excepciones que salen y el código que vemos no lo parece, más bien parece un

ASProtect2 también, en cualquier caso cerramos el mensaje que sale diciendo "Module BatchRep has entry Point outside, bla, bla.." pasamos dichas excepciones y por fin vemos esto en pantalla xDDD



El programa echa a correr desempakado, bouuuuufff¡¡!! después de tanto trabajo hemos obtenido nuestra recompensa.

Y aquí termina por fin esta segunda parte tute, si tengo tiempo estudiaré las limitaciones y haremos la tercera, aunque será opcional puesto que el objetivo era desempakar y está cumplido. He trasteado las funciones del programa y funcionaron bien todas, si alguien encuentra algún fallo/error del programa o mío que porfavor que me lo indique y lo rectificamos.

Hasta aquí el tute de hoy, espero volver pronto con más y mejor como se dice en mi tierra, para cualquier cosa ya sabéis donde contactarme:

syxe05@gmail.com

Un abrazo a TODO CracksLatinoS SIN EXCEPCIÓN, en especial a Ricardo, Red@hawk, Juan José, k0rt, +NCR, Solid, Marciano, Absolom y ZekkT, con quienes he mantenido al menos una vez una conversación acerca de este entretenido packer.. también ThunderPwr a quién le debo un e-mail acerca del mismo tema. Un saludo a todos!